Practica integradora: "El contenedor como proceso" + "Automatización con Docker"

José A. Santos García

Fecha: 03-11-2026

Asignatura: ASO

ÍNDICE

BLOQUE A.....3

1. Identifica el proceso del contenedor en el host.....3

2. Estados y ciclos de vida.....3

3. Proceso vs hilo vs trabajo.....4

4. Señales.....4

5. /proc como registro.....5

6. Seguridad.....6

BLOQUE B.....6

1. Script de automatización.....6

2. Planificación con cron.....7

3. Tarea puntual con at.....7

4. Documentación.....8

a. crontab -l.....8

b. systemctl list-timers.....8

c. Contenidos del log del sistema.....8

BLOQUE A

1. Identifica el proceso del contenedor en el host:

Lo que se muestra con el comando docker inspect es el PID del proceso en el host. Por ello, al hacer un ps -p, podemos ver el mismo PID. Al fin y al cabo, el contenedor no deja de ser un proceso más del host, a pesar de tener procesos propios, como se puede ver con pstree.

```
usuario@ubuntu:~$ docker inspect --format '{{.State.Pid}}' mi-apache-php
1403
usuario@ubuntu:~$ ps -p 1403 -o pid,ppid,cmd
    PID    PPID CMD
   1403    1345 httpd -DFOREGROUND
```

```
usuario@ubuntu:~$ pstree -p 1403
httpd(1403)─┬─httpd(1509)
            ├─httpd(1510)
            ├─httpd(1511)
            ├─httpd(1512)
            └─httpd(1513)
```

2. Estados y ciclos de vida:

Con este comando filtramos, de la información que nos da el comando docker ps sobre el contenedor mi-apache-php, las columnas que nos interesa de la tabla. En este caso filtramos por el nombre, el estado en el que se encuentra y el informe de estado. Hacemos unas pruebas de parar e iniciar de nuevo el contenedor para comprobar los estados. Además, podemos comprobar que cada vez que se vuelve a iniciar el proceso, se asigna un nuevo PID.

```
usuario@ubuntu:~$ docker ps --filter name=mi-apache-php --format 'table {{.Names}}\t{{.Status}}\t
{{.State}}'
NAMES           STATUS          STATE
mi-apache-php   Up 18 minutes   running
usuario@ubuntu:~$ docker stop mi-apache-php
mi-apache-php
usuario@ubuntu:~$ docker ps -a --filter name=mi-apache-php --format 'table {{.Names}}\t{{.Status}
}\t{{.State}}'
NAMES           STATUS                      STATE
mi-apache-php   Exited (0) 13 seconds ago   exited
usuario@ubuntu:~$ docker start mi-apache-php
mi-apache-php
usuario@ubuntu:~$
```

```
usuario@ubuntu:~$ docker inspect --format '{{.State.Pid}}' mi-apache-php
4663
```

3. Proceso vs hilo vs trabajo:

La diferencia entre proceso, hilo y trabajo son:

- Trabajo: son procesos ejecutados en la terminal por el usuario.
- Procesos: son trabajos en términos generales, son tanto los ejecutados por el usuario como por el propio equipo.
- Hilo: son subtrabajos dependientes de un mismo trabajo que trabajan en paralelo para hacer diversas tareas, compartiendo memoria.

```
usuario@ubuntu:~$ docker exec -it mi-apache-php sh
/usr/local/apache2 # sleep 300 &
/usr/local/apache2 # jobs
[1]+  Running                    sleep 300
/usr/local/apache2 # ps
PID   USER     TIME  COMMAND
    1 root      0:00 httpd -DFOREGROUND
    7 www-data  0:00 httpd -DFOREGROUND
    8 www-data  0:00 httpd -DFOREGROUND
    9 www-data  0:00 httpd -DFOREGROUND
   10 www-data  0:00 httpd -DFOREGROUND
   11 www-data  0:00 httpd -DFOREGROUND
   12 root      0:00 sh
   17 root      0:00 sleep 300
   18 root      0:00 ps
/usr/local/apache2 #
```

```
usuario@ubuntu:~$ ps -T -p $(docker inspect --format '{{.State.Pid}}' mi-apache-php)
    PID    SPID TTY          TIME CMD
   4663    4663 ?        00:00:00 httpd
```

4. Señales:

En este ejercicio, pararemos el docker con docker stop y comprobamos que se ha detenido con docker ps -a… Luego, detenemos forzosamente el docker con kill, apareciendo el código 137 confirmando que el docker se ha detenido forzosamente.

```
usuario@ubuntu:~$ docker stop mi-apache-php
mi-apache-php
usuario@ubuntu:~$ docker ps -a --filter name=mi-apache-php --format '{{.Status}}'
Exited (0) 28 seconds ago
usuario@ubuntu:~$ docker start mi-apache-php
mi-apache-php
usuario@ubuntu:~$ docker kill mi-apache-php
mi-apache-php
usuario@ubuntu:~$ docker ps -a --filter name=mi-apache-php --format '{{.Status}}'
Exited (137) 7 seconds ago
usuario@ubuntu:~$
```

5. /proc como registro:

Primero se obtiene el *PID* del proceso principal del contenedor con docker inspect y se guarda en la variable *PID*. Después se lee */proc/$PID/status*, que muestra que es un proceso *httpd* en estado dormido. A continuación, cmdline confirma que el comando con el que se lanzó es *httpd -DFOREGROUND*, es decir, Apache en primer plano, como exige un contenedor para mantenerse vivo. Por último, se intenta listar los descriptores de archivo abiertos en */proc/$PID/fd/*. Con el usuario normal da permiso denegado, porque el proceso pertenece a *root*, así que se repite con *sudo*.

```
usuario@ubuntu:~$ PID=$(docker inspect --format '{{.State.Pid}}' mi-apache-php)
usuario@ubuntu:~$ cat /proc/$PID/status | head -20
Name:   httpd
Umask:  0022
State:  S (sleeping)
Tgid:   4338
Ngid:   0
Pid:    4338
PPid:   4296
TracerPid:      0
Uid:    0       0       0       0
Gid:    0       0       0       0
FDSize: 64
Groups: 0 1 2 3 4 6 10 11 20 26 27
NStgid: 4338    1
NSpid:  4338    1
NSpgid: 4338    1
NSsid:  4338    1
VmPeak:    21496 kB
VmSize:    21496 kB
VmLck:         0 kB
VmPin:         0 kB
usuario@ubuntu:~$ tr '\0' ' ' < /proc/$PID/cmdline; echo
httpd -DFOREGROUND
usuario@ubuntu:~$ ls -l /proc/$PID/fd/
ls: cannot open directory '/proc/4338/fd/': Permission denied
usuario@ubuntu:~$ sudo ls -l /proc/$PID/fd/
[sudo] password for usuario:
Sorry, try again.
[sudo] password for usuario:
total 0
lrwx------ 1 root root 64 nov  3 07:35 0 -> /dev/null
l-wx------ 1 root root 64 nov  3 07:35 1 -> 'pipe:[52593]'
l-wx------ 1 root root 64 nov  3 07:35 2 -> 'pipe:[52594]'
lrwx------ 1 root root 64 nov  3 07:42 3 -> 'socket:[53036]'
lrwx------ 1 root root 64 nov  3 07:42 4 -> 'socket:[53037]'
lr-x------ 1 root root 64 nov  3 07:42 5 -> 'pipe:[53050]'
l-wx------ 1 root root 64 nov  3 07:42 6 -> 'pipe:[53050]'
l-wx------ 1 root root 64 nov  3 07:42 7 -> 'pipe:[52593]'
usuario@ubuntu:~$
```

6. Seguridad:

Iniciamos un docker con nombre “sospechoso” durante una hora (*sleep 3600*). Después, comprobamos con *docker ps -a*, escogiendo que solo aparezca el nombre, la imagen y el estado, para comprobar que se ha levantado. Posteriormente, buscamos con *ps aux* el proceso, filtrando por el comando *sleep*, y lo eliminamos con el comando *docker rm -f* para forzar dicha eliminación.

```
usuario@ubuntu:~$ docker run -d --name sospechoso alpine sleep 3600
eef37c9eedf29e5dbfd4b893cbbc91d0a11233697ce2d4232b188eabc2c696f3
usuario@ubuntu:~$ docker ps -a --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
NAMES                IMAGE                 STATUS
sospechoso           alpine                Up 39 seconds
romantic_matsumoto   ubuntu:20.04          Exited (0) 4 days ago
mi-apache-php        stack-httpd-php-web   Up 10 minutes
mi-mariadb           mariadb:10.11         Up 17 minutes
web-personalizada2   mi_web:1.0            Exited (0) 11 days ago
mi-ubuntu            ubuntu:20.04          Exited (0) 2 weeks ago
optimistic_pascal    ubuntu:20.04          Exited (0) 2 weeks ago
epic_heisenberg      hello-world           Exited (0) 2 weeks ago
web-vol              httpd:latest          Exited (0) 2 weeks ago
web-bind             httpd:latest          Exited (0) 2 weeks ago
web2                 httpd                 Exited (0) 2 weeks ago
web1                 httpd                 Exited (0) 2 weeks ago
web                  httpd                 Exited (0) 2 weeks ago
lunesUbuntu          ubuntu:20.04          Exited (0) 2 weeks ago
goofy_maxwell        httpd                 Exited (0) 2 weeks ago
goofy_bose           ubuntu:20.04          Exited (127) 3 weeks ago
eager_engelbart      ubuntu                Exited (130) 3 weeks ago
unruffled_kilby      ubuntu                Exited (0) 3 weeks ago
wonderful_bhabha     ubuntu:20.04          Exited (0) 3 weeks ago
hungry_volhard       ubuntu                Exited (0) 3 weeks ago
elastic_hellman      hello-world           Exited (0) 3 weeks ago
naughty_mccarthy     hello-world           Exited (0) 3 weeks ago
usuario@ubuntu:~$ ps aux | grep -i sleep grep -v grep
grep: grep: No such file or directory
grep: grep: No such file or directory
usuario@ubuntu:~$ ps aux | grep -i sleep | grep -v grep
root        4646  0.0  0.0   1620     4 ?        Ss   07:44   0:00 sleep 3600
usuario@ubuntu:~$ docker rm -f sospechoso
sospechoso
```

BLOQUE B

1. Script de automatización:

Se crean las carpetas correspondientes de */bin* y */logs* en el directorio de */home/usuario* y creamos el script dentro del *~/bin*, asignándole permisos de ejecución. El script *check_stack.sh* define el log (*~/logs/check_stack.log*), el directorio del stack y el nombre del contenedor *mi-apache-php*. Primero comprueba si existe el usuario *webadmin* y, si no, lo crea con su home y *shell bash*, anotándolo en el log. Después verifica con *docker ps* si el contenedor está en ejecución y registra *[OK]* si es así; si no, registra *[WARN]* y lanza *docker compose up -d* desde el directorio del stack. Por último, según el código de salida (*$?*), escribe en el log *[OK]* si el arranque fue correcto o *[ERROR]* si falló.

```
usuario@ubuntu:~$ mkdir -p ~/bin
usuario@ubuntu:~$ mkdir -p ~/logs
usuario@ubuntu:~$ nano ~/bin/check_stack.sh
usuario@ubuntu:~$ chmod +x ~/bin/check_stack.sh
```

```bash
#!/bin/bash
# check_stack.sh - Verifica y arranca el stack httpd+php si es necesario
# Autor: alumno ASIR
# Fecha: $(date +%Y-%m-%d)
LOG=~/logs/check_stack.log
STACK_DIR=~/stack-httpd-php
CONTAINER=mi-apache-php
# Crear usuario webadmin si no existe (RA3 e)
if ! id -u webadmin >/dev/null 2>&1; then
 sudo useradd -m -s /bin/bash webadmin
 echo "$(date '+%F %T') [INFO] Usuario webadmin creado" >> "$LOG"
fi
# Comprobar si el contenedor está corriendo
if docker ps --filter "name=$CONTAINER" --filter "status=running" | grep -q "$CONTAINER";
then
 echo "$(date '+%F %T') [OK] $CONTAINER en ejecución" >> "$LOG"
else
 echo "$(date '+%F %T') [WARN] $CONTAINER caído. Arrancando stack..." >> "$LOG"
 cd "$STACK_DIR" && docker compose up -d >> "$LOG" 2>&1
 if [ $? -eq 0 ]; then
 echo "$(date '+%F %T') [OK] Stack arrancado correctamente" >> "$LOG"
 else
 echo "$(date '+%F %T') [ERROR] Fallo al arrancar el stack" >> "$LOG"
 fi
fi
```

2. Planificación con cron:

Planificamos la ejecución del script con crontab, metiéndonos en el fichero con *crontab -e*. Comprobamos que el crontab está guardado con *crontab -l*.

```
usuario@ubuntu:~$ crontab -e
crontab: installing new crontab
usuario@ubuntu:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
#
# Verificar el stack httpd+php cada 5 minutos
*/5 * * * * /home/usuario/bin/check_stack.sh
```

3. Tarea puntual con at:

Se programa con *at* una tarea única que ejecutará *cd ~/stack-httpd-php && docker compose down* dentro de 2 minutos; el sistema avisa de que se usará */bin/sh* y asigna el trabajo número 1 para las 08:08:00. Después, *atq* lista la cola de tareas pendientes y confirma que ese trabajo 1 está programado para el usuario. En la segunda captura, *at -c* 1 muestra el contenido completo del trabajo donde, al final del archivo, aparece un *cd /home/usuario* de comprobación (con salida de error si el directorio fuera inaccesible) y, por último, el comando programado, que detendrá y eliminará el stack.

```
usuario@ubuntu:~$ echo "cd ~stack-httpd-php && docker compose down" | at now + 2 minutes
warning: commands will be executed using /bin/sh
job 1 at Tue Nov  3 08:08:00 2026
usuario@ubuntu:~$ atq
1       Tue Nov  3 08:08:00 2026 a usuario
```

```
usuario@ubuntu:~$ at -c 1
#!/bin/sh
# atrun uid=1000 gid=1000
# mail usuario 0
umask 2
SESSION_MANAGER=local/ubuntu:@/tmp/.ICE-unix/3104,unix/ubuntu:/tmp/.ICE-unix/3104; export SESSION_MANAGER
QT_ACCESSIBILITY=1; export QT_ACCESSIBILITY
COLORTERM=truecolor; export COLORTERM
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg; export XDG_CONFIG_DIRS
XDG_MENU_PREFIX=gnome-; export XDG_MENU_PREFIX
GNOME_DESKTOP_SESSION_ID=this-is-deprecated; export GNOME_DESKTOP_SESSION_ID
GNOME_SHELL_SESSION_MODE=ubuntu; export GNOME_SHELL_SESSION_MODE
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh; export SSH_AUTH_SOCK
XMODIFIERS=@im=ibus; export XMODIFIERS
DESKTOP_SESSION=ubuntu; export DESKTOP_SESSION
SSH_AGENT_PID=3066; export SSH_AGENT_PID
GTK_MODULES=gail:atk-bridge; export GTK_MODULES
PWD=/home/usuario; export PWD
LOGNAME=usuario; export LOGNAME
XDG_SESSION_DESKTOP=ubuntu; export XDG_SESSION_DESKTOP
XDG_SESSION_TYPE=x11; export XDG_SESSION_TYPE
GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1; export GPG_AGENT_INFO
XAUTHORITY=/run/user/1000/gdm/Xauthority; export XAUTHORITY
WINDOWPATH=2; export WINDOWPATH
HOME=/home/usuario; export HOME
USERNAME=usuario; export USERNAME
IM_CONFIG_PHASE=1; export IM_CONFIG_PHASE
LANG=es_ES.UTF-8; export LANG
LS_COLORS=rs=0:di=01\;34:ln=01\;36:mh=00:pi=40\;33:so=01\;35:do=01\;35:bd=40\;33\;01:cd=40\;33\;01:or=40\;31\;01:mi=00:su=37\;41:sg=30\;43:ca=30\
;41:tw=30\;42:ow=34\;42:st=37\;44:ex=01\;32:\*.tar=01\;31:\*.tgz=01\;31:\*.arc=01\;31:\*.arj=01\;31:\*.taz=01\;31:\*.lha=01\;31:\*.lz4=01\;31:\*.
lzh=01\;31:\*.lzma=01\;31:\*.tlz=01\;31:\*.txz=01\;31:\*.tzo=01\;31:\*.t7z=01\;31:\*.zip=01\;31:\*.z=01\;31:\*.dz=01\;31:\*.gz=01\;31:\*.lrz=01\;
31:\*.lz=01\;31:\*.lzo=01\;31:\*.xz=01\;31:\*.zst=01\;31:\*.tzst=01\;31:\*.bz2=01\;31:\*.bz=01\;31:\*.tbz=01\;31:\*.tbz2=01\;31:\*.tz=01\;31:\*.d
eb=01\;31:\*.rpm=01\;31:\*.jar=01\;31:\*.war=01\;31:\*.ear=01\;31:\*.sar=01\;31:\*.rar=01\;31:\*.alz=01\;31:\*.ace=01\;31:\*.zoo=01\;31:\*.cpio=0
1\;31:\*.7z=01\;31:\*.rz=01\;31:\*.cab=01\;31:\*.wim=01\;31:\*.swm=01\;31:\*.dwm=01\;31:\*.esd=01\;31:\*.jpg=01\;35:\*.jpeg=01\;35:\*.mjpg=01\;35
:\*.mjpeg=01\;35:\*.gif=01\;35:\*.bmp=01\;35:\*.pbm=01\;35:\*.pgm=01\;35:\*.ppm=01\;35:\*.tga=01\;35:\*.xbm=01\;35:\*.xpm=01\;35:\*.tif=01\;35:\*
.tiff=01\;35:\*.png=01\;35:\*.svg=01\;35:\*.svgz=01\;35:\*.mng=01\;35:\*.pcx=01\;35:\*.mov=01\;35:\*.mpg=01\;35:\*.mpeg=01\;35:\*.m2v=01\;35:\*.m
kv=01\;35:\*.webm=01\;35:\*.ogm=01\;35:\*.mp4=01\;35:\*.m4v=01\;35:\*.mp4v=01\;35:\*.vob=01\;35:\*.qt=01\;35:\*.nuv=01\;35:\*.wmv=01\;35:\*.asf=0
1\;35:\*.rm=01\;35:\*.rmvb=01\;35:\*.flc=01\;35:\*.avi=01\;35:\*.fli=01\;35:\*.flv=01\;35:\*.gl=01\;35:\*.dl=01\;35:\*.xcf=01\;35:\*.xwd=01\;35:\
*.yuv=01\;35:\*.cgm=01\;35:\*.emf=01\;35:\*.ogv=01\;35:\*.ogx=01\;35:\*.aac=00\;36:\*.au=00\;36:\*.flac=00\;36:\*.m4a=00\;36:\*.mid=00\;36:\*.mid
i=00\;36:\*.mka=00\;36:\*.mp3=00\;36:\*.mpc=00\;36:\*.ogg=00\;36:\*.ra=00\;36:\*.wav=00\;36:\*.oga=00\;36:\*.opus=00\;36:\*.spx=00\;36:\*.xspf=00
\;36:; export LS_COLORS
XDG_CURRENT_DESKTOP=ubuntu:GNOME; export XDG_CURRENT_DESKTOP
VTE_VERSION=6003; export VTE_VERSION
GNOME_TERMINAL_SCREEN=/org/gnome/Terminal/screen/f5ff7c5b_ae91_43e2_8123_769189871757; export GNOME_TERMINAL_SCREEN
INVOCATION_ID=9a9599dd2dc644adbe65108c689e0269; export INVOCATION_ID
MANAGERPID=2924; export MANAGERPID
LESSCLOSE=/usr/bin/lesspipe\ %s\ %s; export LESSCLOSE
XDG_SESSION_CLASS=user; export XDG_SESSION_CLASS
LESSOPEN=\|\ /usr/bin/lesspipe\ %s; export LESSOPEN
USER=usuario; export USER
GNOME_TERMINAL_SERVICE=:1.67; export GNOME_TERMINAL_SERVICE
SHLVL=0; export SHLVL
QT_IM_MODULE=ibus; export QT_IM_MODULE
XDG_RUNTIME_DIR=/run/user/1000; export XDG_RUNTIME_DIR
JOURNAL_STREAM=9:42673; export JOURNAL_STREAM
XDG_DATA_DIRS=/usr/share/ubuntu:/usr/local/share/:/usr/share/:/var/lib/snapd/desktop; export XDG_DATA_DIRS
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin; export PATH
GDMSESSION=ubuntu; export GDMSESSION
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus; export DBUS_SESSION_BUS_ADDRESS
cd /home/usuario || {
        echo 'Execution directory inaccessible' >&2
        exit 1
}
cd ~stack-httpd-php && docker compose down
```

4. Documentación:

Debido a que el docker no ha sido usado antes, no tiene instalado ninguno de los comandos que se van a usar, por lo que hay que instalarlos. Sin embargo, a pesar de haberlos instalado, los comandos me dan error salvo el de crontab -l, que indica que no hay ningún crontab para el usuario root.

a. crontab -l

```
usuario@ubuntu:~$ docker exec -it romantic_matsumoto crontab -l
no crontab for root
```

b. systemctl list-timers

c. Contenidos del log del sistema

Conclusión

En esta práctica he podido comprobar que un contenedor de Docker no es más que un proceso que se ejecuta en el host, cuyo *PID* se puede identificar y monitorizar mediante herramientas del sistema como *ps, pstree* o consultando el directorio */proc*. He analizado sus ciclos de vida, las señales de control y parada y he aplicado técnicas de seguridad para la gestión de procesos. Además, en el apartado de automatización, he desarrollado un script de supervisión y lo he integrado en el sistema mediante planificadores como *cron* y *at*, lo que me ha permitido automatizar el mantenimiento y la recuperación de un *stack* con *Docker Compose*.

Fuentes consultadas